

multGen.py

×

```
1  # WRITE YOUR CODE HERE
2  import os
3  import openai
4  import requests
5  import secret
6
7  openai.api_key='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'
8
9  # Set the prompts
10 prompts = ["robot dog in a lab", "robot dog exploring the city", "robot dog v
11
12 # Generate and save the images
13 for i, prompt in enumerate(prompts):
14     response = openai.Image.create(
15         prompt=prompt,
16         n=1,
17         size="256x256"
18     )
19     \
```



24% (6:1)

Terminal

×

```
* Canonical Livepatch is available for installation.
- Reduce system reboots and improve kernel security. Activate at:
https://ubuntu.com/livepatch
*
* Welcome to the Codio Terminal!
*
* https://docs.codio.com/develop/develop/ide/boxes/overview
*
* Your Codio Box domain is: analogorlando-edgarbuenos.codio.io
*
Last login: Mon Feb 27 20:05:42 2023 from 192.168.11.179
codio@analogorlando-edgarbuenos:~/workspace$ python multGen.py
codio@analogorlando-edgarbuenos:~/workspace$ ls
async.py      imageGen.py      image_name.jpg      reset.py          sync.py
async.sh      image_name1.jpg  multGen.py          robot_dog_journey_1.jpg  sync.sh
bash.sh       image_name2.jpg  ninja_cat_adventure_0.png  robot_dog_journey_2.jpg  test1.j
exerc.py      image_name3.jpg  __pycache__         robot_dog_journey_3.jpg  test2.j
exercTest.py  image_name-copy.jpg  README.md          secret.py         test3.j
codio@analogorlando-edgarbuenos:~/workspace$
```

Guide

Collapse

Chaining Multiple API Calls to Create a Sequence of Images with D...

```
import os
import openai
import requests
import secret

openai.api_key=secret.api_key

# Set the prompts
prompts = ["robot dog in a lab", "robot dog exploring the city", "robot dog v

# Generate and save the images
for i, prompt in enumerate(prompts):
    response = openai.Image.create(
        prompt=prompt,
        n=1,
        size="256x256"
    )

# Get the image URL from the response
image_url = response['data'][0]['url']

# Download and save the image
img_data = requests.get(image_url).content
with open(f"robot_dog_journey_{i+1}.jpg", 'wb') as handler:
    handler.write(img_data)
```

TRY IT!

Command executed

After clicking the `Try it` make sure to be give it a few more seconds so that the images can be generated even though we see the message that the code was executed.

Click here to refresh your image 1

Click here to refresh your image 2

Click here to refresh your image 3

In this example, we first define a list of prompts. Each prompt represents a different stage of the "robot dog's journey".

We then loop over each prompt in the list. For each prompt, we call the DALL-E API to generate an image that corresponds to that stage of the journey. We specify prompt as the prompt, `n=1` to generate one image per prompt, and `size="256x256"` to specify the size of the generated image.

From the API response, we extract the URL of the generated image. We then use the `requests` library to download the image from this URL.

Finally, we save each image with a unique filename that includes the index of the prompt in the list. This allows us to view the images in the order they were generated, effectively seeing the robot dog's journey unfold in sequence.

By chaining these API calls together, we can generate a sequence of images that tell a visually compelling story. This approach can be extended and adapted to generate sequences of images for a wide range of applications.

What is one benefit of chaining API calls when using OpenAI's DALL-E API?

- ☒ It allows us to create a sequence of images that progressively change.
- ☐ It reduces the complexity of the outputs.
- ☐ It allows us to generate a single image.
- ☐ It limits the creativity of the outputs.

Chaining API calls allows us to generate a sequence of images based on a set of prompts. This can be used for storytelling, progressive image generation, and exploring different variations of the same concept.

Check It!

Next